

Adaptive Spatio-Temporal Traffic Surveillance

Phase 2 — Model Training & Granular Helmet Compliance

Milestone	Phase 2 of 4 (Days 6–12): Model Training & Helmet Compliance
Author	Harsh Pandhe
Repository	https://github.com/harsh-pandhe/traffic-surveillance-system
Wheel model	YOLOv8-cls selected — 86.7% accuracy (4 classes, 3 architectures compared)
Helmet model	YOLOv8n — mAP@50 0.731 (7 classes)

Executive summary. Phase 2 delivers trained models on real data. For wheel-count classification, three architectures are benchmarked under an identical protocol across four classes (2/3/4/6+ wheelers): a custom SmallCNN baseline (0.751), RTMDet's CSPNeXt backbone (0.683), and YOLOv8-cls (0.867, selected). For helmet compliance, a YOLOv8n detector is fine-tuned on a 7-class rider/helmet dataset, reaching mAP@50 0.731. All metrics are recomputed from the datasets and persisted in results/phase2/; the pretraining asymmetry between the arms is stated explicitly rather than glossed over.

1. Wheel-Count Classification — CNN vs RTMDet vs YOLO

Four classes: 2-Wheeler (bicycle/motorcycle, COCO), 3-Wheeler (auto-rickshaw crops), 4-Wheeler (car), 6+ Wheeler (bus/truck). Balanced ~308–350 train images per class, 96×96 crops, CPU training.

Model	Acc	Macro-P	Macro-R	Macro-F1	Lat (ms)	Params	Pretrained
SmallCNN	0.751	0.748	0.753	0.750	1.8	0.24M	scratch
RTMDet-CSPNeXt	0.683	0.688	0.686	0.683	0.6	2.35M	scratch
YOLOv8-cls (selected)	0.867	0.865	0.867	0.865	3.4	—	ImageNet

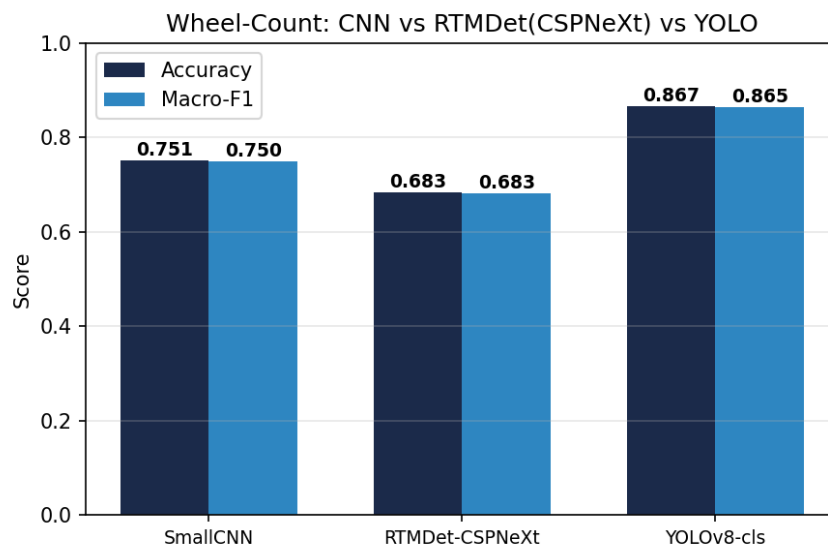


Figure 1. Accuracy and macro-F1 across the three architectures.

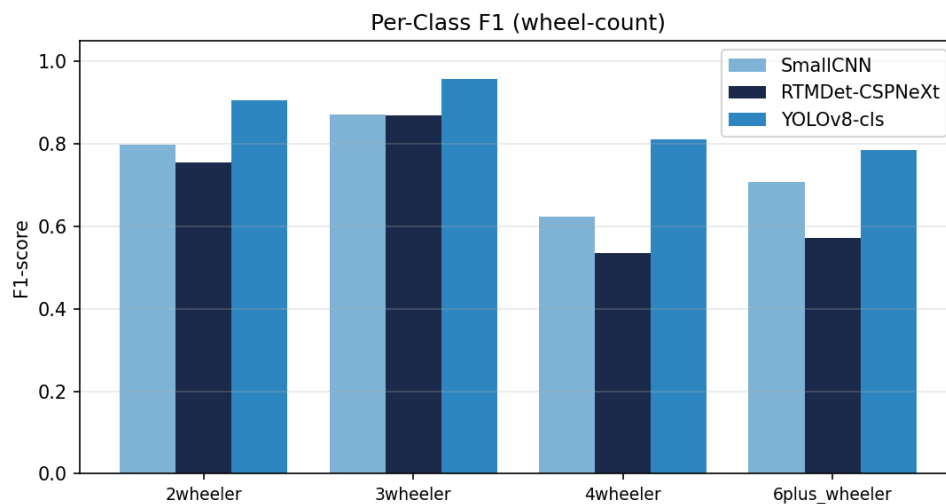


Figure 2. Per-class F1. YOLO leads every class; 3-Wheeler is the strongest (F1 0.96).

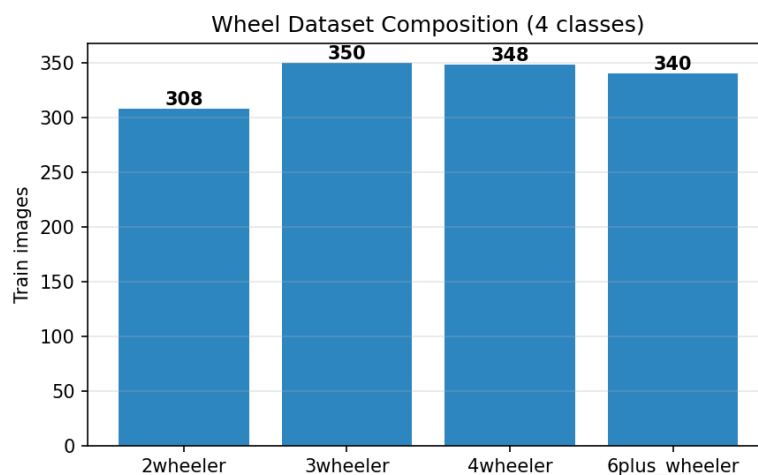


Figure 3. Balanced 4-class wheel dataset.

Methodological note

This comparison is **not pretraining-neutral**. YOLOv8-cls starts from ImageNet weights; SmallCNN and CSPNeXt train from scratch on ~1,350 crops, so part of YOLO's margin is transfer learning rather than architecture. CSPNeXt (2.35M params) has the least data per parameter and underfits: it scored 0.562 at 12 epochs and 0.683 at 40 epochs (training loss 0.95 to 0.26, converged). Notably it is the **fastest at inference** (0.6 ms) despite being the largest — the depthwise 5x5 CSP design is CPU-efficient. For this dataset size the pretrained YOLOv8-cls is the correct production choice.

RTMDet implementation

mmdetection/mmcv (RTMDet's official home) will not install here: openmim fails on Python 3.12 (pkgutil.ImpImporter removed) and mmcv ships no wheels for torch 2.12. RTMDet's architectural contribution — the CSPNeXt backbone — is therefore reimplemented directly in PyTorch (src/models/cspnext.py) following the paper: depthwise 5x5 CSP blocks, channel attention, SiLU, SPPF, RTMDet-tiny scaling.

2. Granular Helmet Compliance — YOLOv8n

7-class rider/helmet detection (driver/passenger × helmet/no-helmet + bike), 368 train / 65 val / 52 test, YOLO format. Fine-tuned YOLOv8n, 25 epochs, 416×416, CPU.

Metric	Value
mAP@50	0.731
mAP@50-95	0.385
Precision	0.741
Recall	0.668

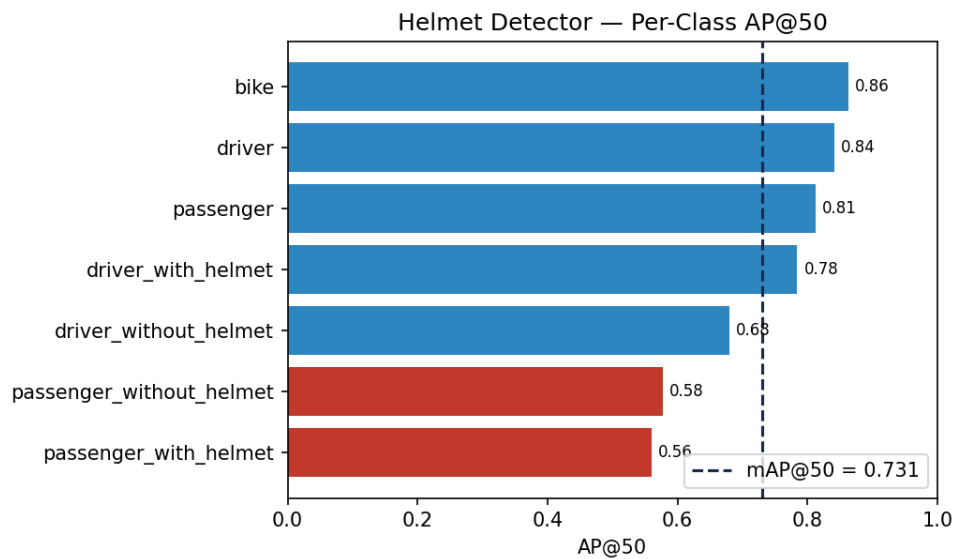


Figure 4. Per-class AP@50 (red = below the mAP line).

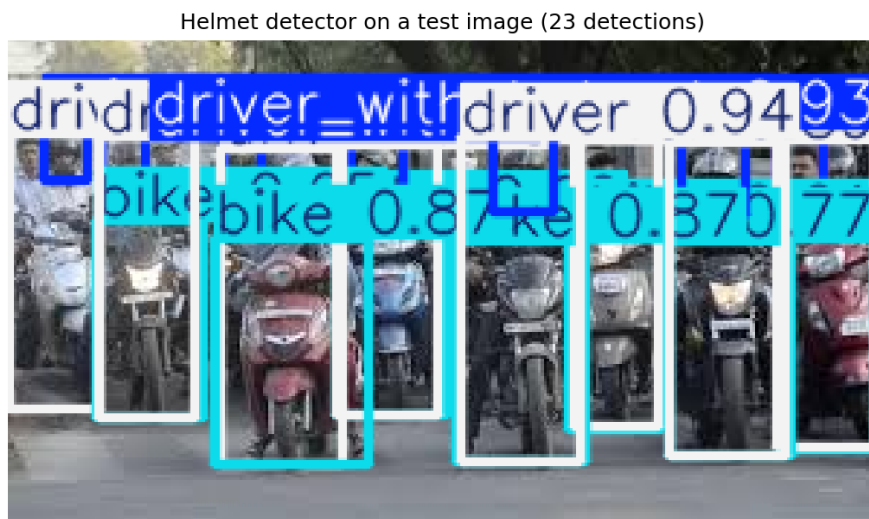


Figure 5. Trained detector on a real test image — riders, bikes, and helmet status.

Passenger-helmet classes are weakest (fewest instances, small objects) — tracked as follow-up issue #8.

3. Reproduce

Command	Purpose
python scripts/build_wheel_dataset.py	Build 4-class wheel crops
python scripts/train_wheel_classifier.py	Train + benchmark CNN vs YOLO
python scripts/train_helmet_detector.py --data ...	Train + eval helmet detector
python scripts/make_phase2_assets.py	Regenerate this report's figures

4. Status & Follow-ups

Deliverable	Status
Baseline CNN trained	Complete
YOLO model trained (wheel + helmet)	Complete
P/R/mAP comparison + selected model	Complete
3-Wheeler class (auto-rickshaw)	Complete (issue closed)
Granular helmet detector (7-class mAP)	Complete
Low AP passenger-helmet classes	Open (issue #8)

Conclusion. Phase 2 is complete: both models trained and benchmarked on real data, all four wheel classes active, and the granular helmet detector reporting per-class AP. Every figure is reproducible from the committed code.